

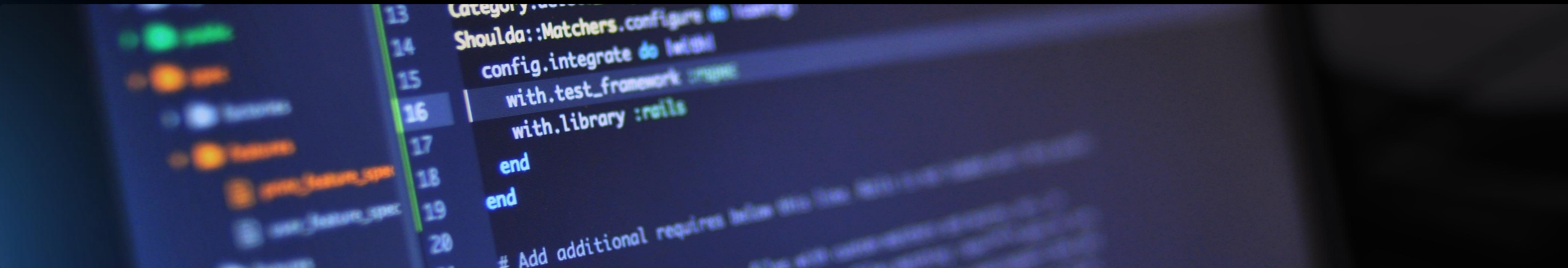


Python for beginners with applications

*Write code from scratch in a clear & concise way, through a complete basic course.
From beginners to intermediate.*

Lesson 01:

- Introduction & Background
- First installation
- Python Programs Editors (Console, text & IDE)
- Write your first program
- Basic Data Types
- Input - Output
- Operations on strings & numbers
- Data flows, Conditions & Loops
- Mutable vs Immutable Data Types



Materials needed

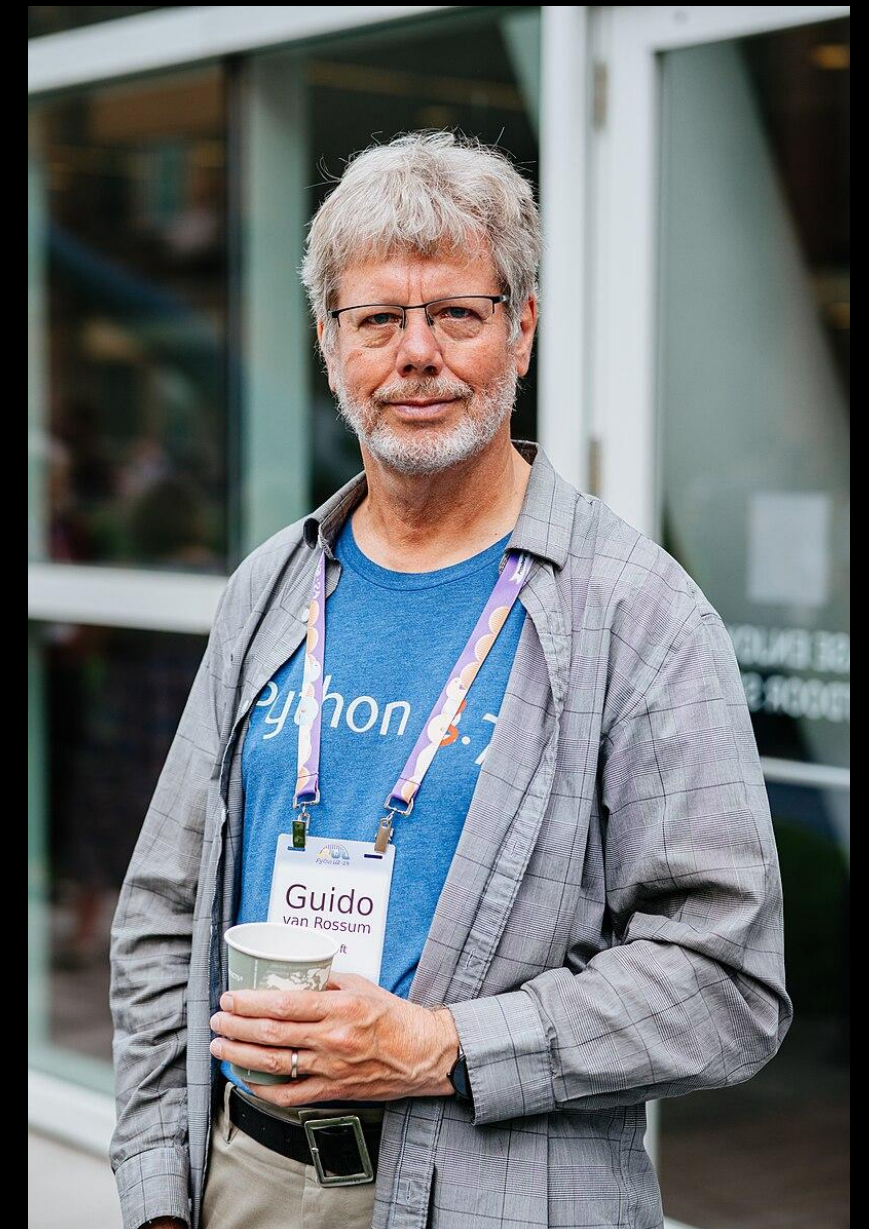
(Python Course 01)

- No special programming background needed.
 - Internet access
 - A clean installation of Python (see <https://www.python.org> Download & install the latest version)
 - A text editor (notepad, notepad++, Sublimetext, ...)
 - Visual Studio Code, Pycharm or Anaconda
 - Jupyternotes or Jupyterlabs
-
- The presentation and the code used will be available on https://github.com/Abdou-fi/python/tree/main/2025-2026/Course_01

Guido van Rossum began working on Python in the late 1980s as a successor to the **ABC** programming language and first released it in 1991 as Python 0.9.0. Python 2.0 was released in 2000. Python 3.0, released in 2008, was a major revision not completely **backward-compatible** with earlier versions. Python 2.7.18, released in 2020, was the last release of Python 2.

Python consistently ranks as one of the most popular programming languages and has gained widespread use in the **machine learning** community.

Learning Scientific Programming with Python is intended to help scientists and engineers learn version 3 the Python programming language and its associated NumPy, SciPy, pandas and Matplotlib libraries. No prior programming experience or scientific knowledge in any particular field is assumed. However, familiarity with some mathematical concepts such as trigonometry, complex numbers and basic calculus is helpful to follow the examples and exercises.

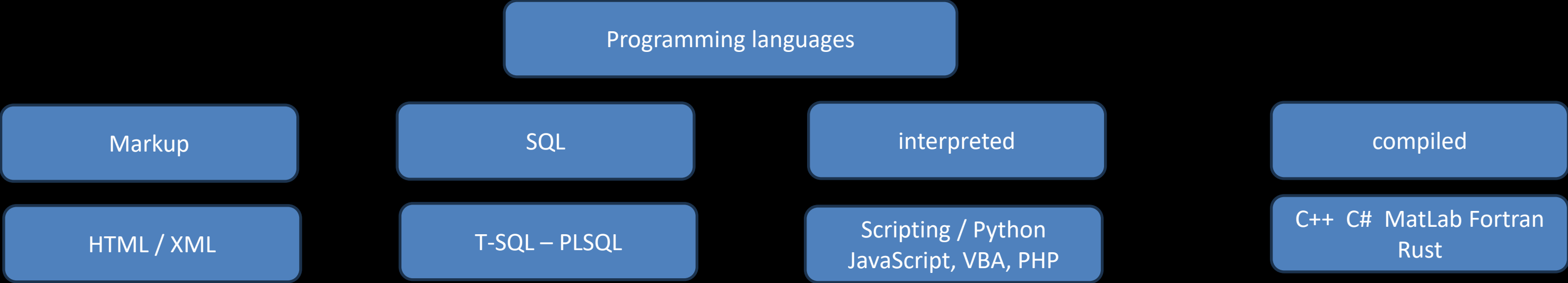


Python is a **high-level, general-purpose** programming language. Its design philosophy emphasizes **code readability** with the use of **significant indentation**.

Python is **dynamically type-checked** and **garbage-collected**. It supports multiple **programming paradigms**, including **structured** (particularly **procedural**), **object-oriented** and **functional programming**. It is often described as a "**batteries included**" language due to its comprehensive **standard library**.

<https://docs.python.org/3/>

It is an interpreted language (JIT, machine code)



Compiled code is translated into machine code before execution, resulting in faster performance and an executable file, while interpreted code is translated line by line as it runs, offering easier debugging and portability. Compiled programs run faster but lack portability, as they are platform-specific, whereas interpreted programs run on any machine with the correct interpreter but can be slower.

Feature	Compiled	Interpreted
Translation	Entire source code translated to machine code before execution.	Source code translated and executed line-by-line at runtime.
Performance	Generally faster because it's pre-compiled machine code	Generally slower due to the overhead of translating each line during execution.
Portability	Lower portability; the executable is specific to the operating system or CPU.	Higher portability; the source code can run on any machine with a compatible interpreter.
Error Detection	Errors are caught during compilation, before the program runs.	Errors are caught during runtime, as the interpreter finds them line by line.
Development	Longer development cycle due to the compile-link-run steps.	Faster development cycle; easy to test and edit code on the go.
Code Distribution	Provides the executable file, keeping the source code private.	Requires the source code to be distributed, making it more public.
Examples	C, C++, Rust	Python, JavaScript, VBA

Nov 2025	Nov 2024	Change	Programming Language		Ratings	Change
1	1			Python	23.37%	+0.52%
2	4	▲		C	9.68%	+0.67%
3	2	▼		C++	8.95%	-1.69%
4	3	▼		Java	8.54%	-1.06%
5	5			C#	7.65%	+2.67%
6	6			JavaScript	3.42%	-0.29%
7	9	▲		Visual Basic	3.31%	+1.36%
8	11	▲		Delphi/Object Pascal	2.06%	+0.58%
9	27	▲▲		Perl	1.84%	+1.16%
10	10			SQL	1.80%	-0.14%
11	7	▼▼		Go	1.72%	-0.63%
12	18	▲▲		R	1.67%	+0.65%

REASONS WHY _ THE MASSIVE POPULARITY OF PYTHON WILL REMAIN INTACT IN THE FUTURE

- Supports Multiple Programming Paradigms
- Doesn't Require Programmers to Write Lengthy Code
- Provides a Comprehensive Standard Library
- Effectuates Web Application Development
- Facilitates Development of High-Quality GUI, Scientific and Numeric applications
- Simplifies Prototyping of Applications
- Can also be used for Mobile App Development
- Open Source
- Easy to learn
- Versatile
- Large community
- High demand
- **Excel**

Java

```
1 File dir = new File("."); // get current directory
2 File fin = new File(
3     dir.getCanonicalPath() + File.separator + "Code.txt"
4 );
5
6 FileInputStream fis = new FileInputStream(fin);
7
8 // Construct the BufferedReader object
9 BufferedReader in = new BufferedReader(
10     new InputStreamReader(fis));
11
12 String aLine = null;
13 while ((aLine = in.readLine()) != null) {
14     // //Process each line, here we count empty lines
15     if (aLine.trim().length() == 0) {}
16 }
17
18 // do not forget to close the buffer reader
19 in.close();
```

Python

```
1 my_file = open("/home/xiaoran/Desktop/test.txt")
2
3 print(my_file.read())
4 my_file.close()
```

C++ "HelloWorld"

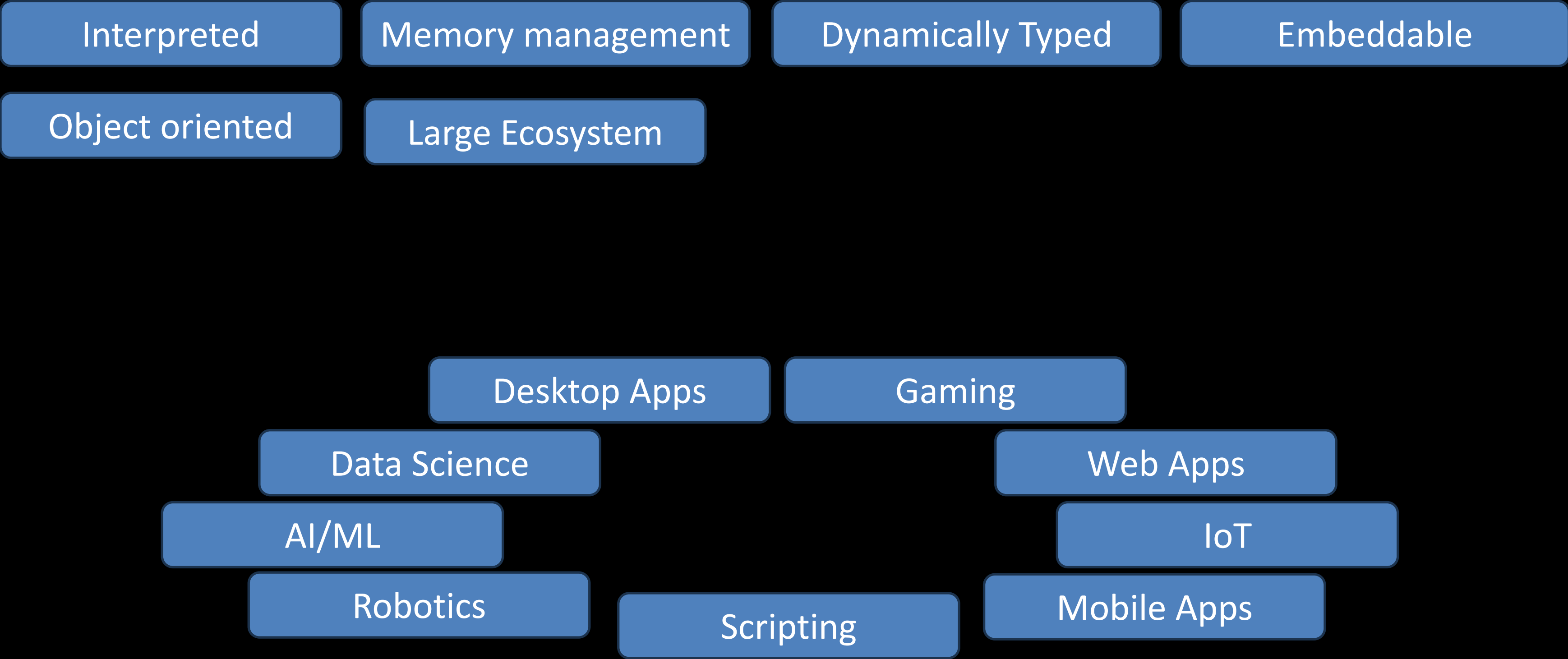
```
#include <iostream>
int main() {
    cout<<"Hello World";
    return 0;
}
```

JAVA "Hello World"

```
class HelloWorld
{
    public static void main(String[] args) {
        system.out.println("Hello World");
    }
}
```

Python "Hello World"

```
print("Hello World")
```



Python	+	NumPy	=	Arrays + Matrices
Python	+	Pandas	=	Data Manipulation
Python	+	Scikit-Learn	=	Machine Learning
Python	+	Tensorflow	=	Deep Learning
Python	+	Matplotlib	=	Data Visualisation
Python	+	Seaborn	=	Adv. Visualisation
Python	+	Flask	=	Web Development
Python	+	Django	=	Web Development
Python	+	Pygame	=	Game Development
Python	+	Kivy	=	Mobile App development

Ressources (Tutorials):

https://www.youtube.com/watch?v=nLRL_NcnK-4 **(Harvard)**

<https://ocw.mit.edu/courses/6-0001-introduction-to-computer-science-and-programming-in-python-fall-2016/> **(MIT)**

<https://www.youtube.com/watch?v=LamjAFnybo0>

<https://www.youtube.com/watch?v=qgUd24sQjBc&list=WL&index=5>

<https://www.youtube.com/watch?v=eWRfhZUzrAc&list=PLWKjhJtqVAbnqBxcdjVGgT3uVR10bzTEB>

https://www.youtube.com/watch?v=ix9cRaBkVe0&list=PLZPZq0r_RZON03iKBjYOsOKr1-TD7z2IH

<https://www.youtube.com/watch?v=9OgOay9bRU8>

<https://www.youtube.com/watch?v=37YPyDx7Zj8>

Python

PSF

Docs

PyPI

Jobs

Community



Donate



Search

GO

Socialize

About

Downloads

Documentation

Community

Success Stories

News

Events

```
# Python 3: Fibonacci sequence
>>> def fib(n):
>>>     a, b = 0, 1
>>>     while a < n:
>>>         print(a, end=' ')
>>>         a, b = b, a+b
>>> fib(1000)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987
```

All releases

Source code

Windows

macOS

Android

Other Platforms

License

Alternative Implementations

Download for Windows

Python 3.14.0

Note that Python 3.9+ cannot be used on Windows 7 or earlier.

Not the OS you are looking for? Python can be used on many operating systems and environments.

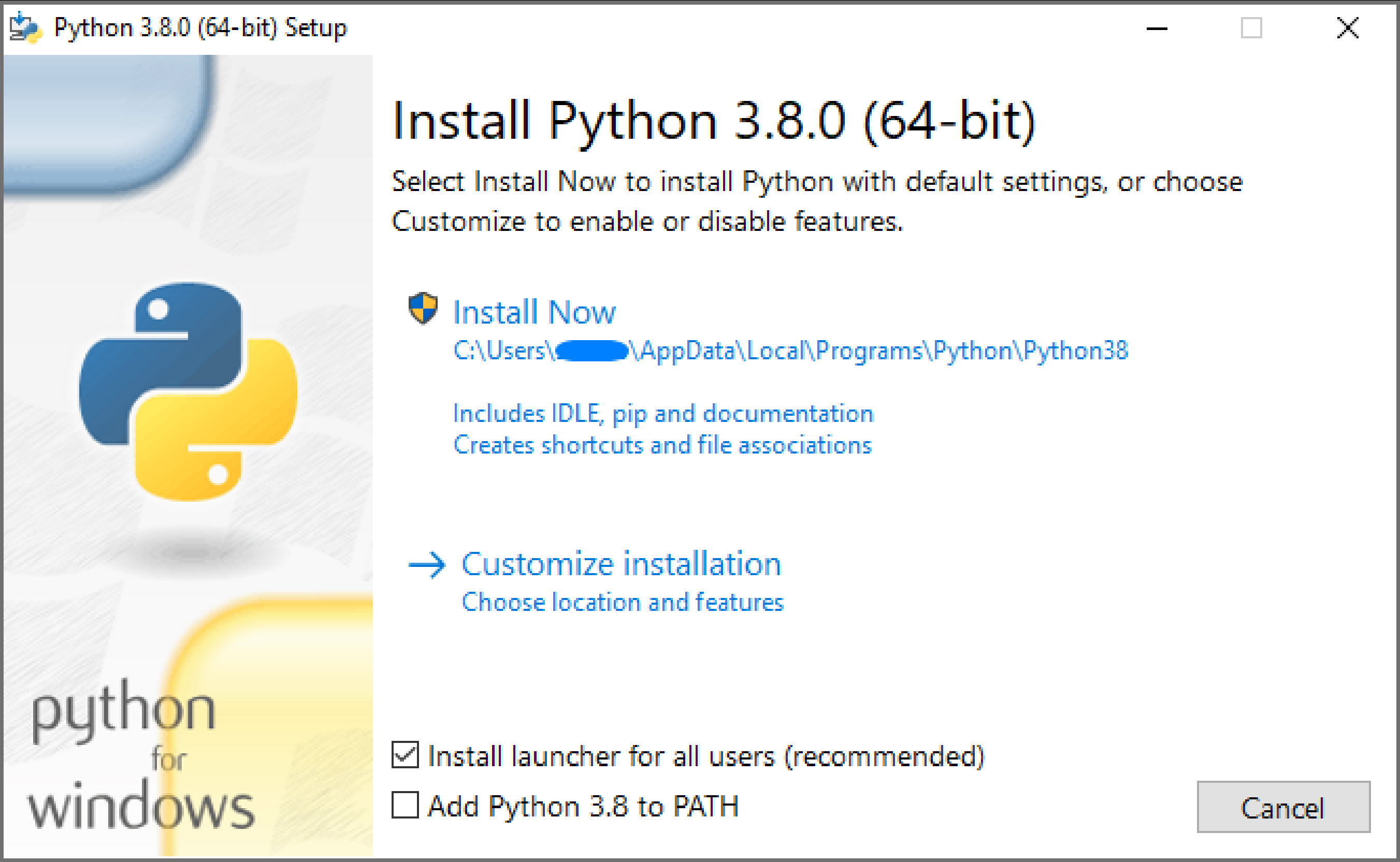
[View the full list of downloads.](#)

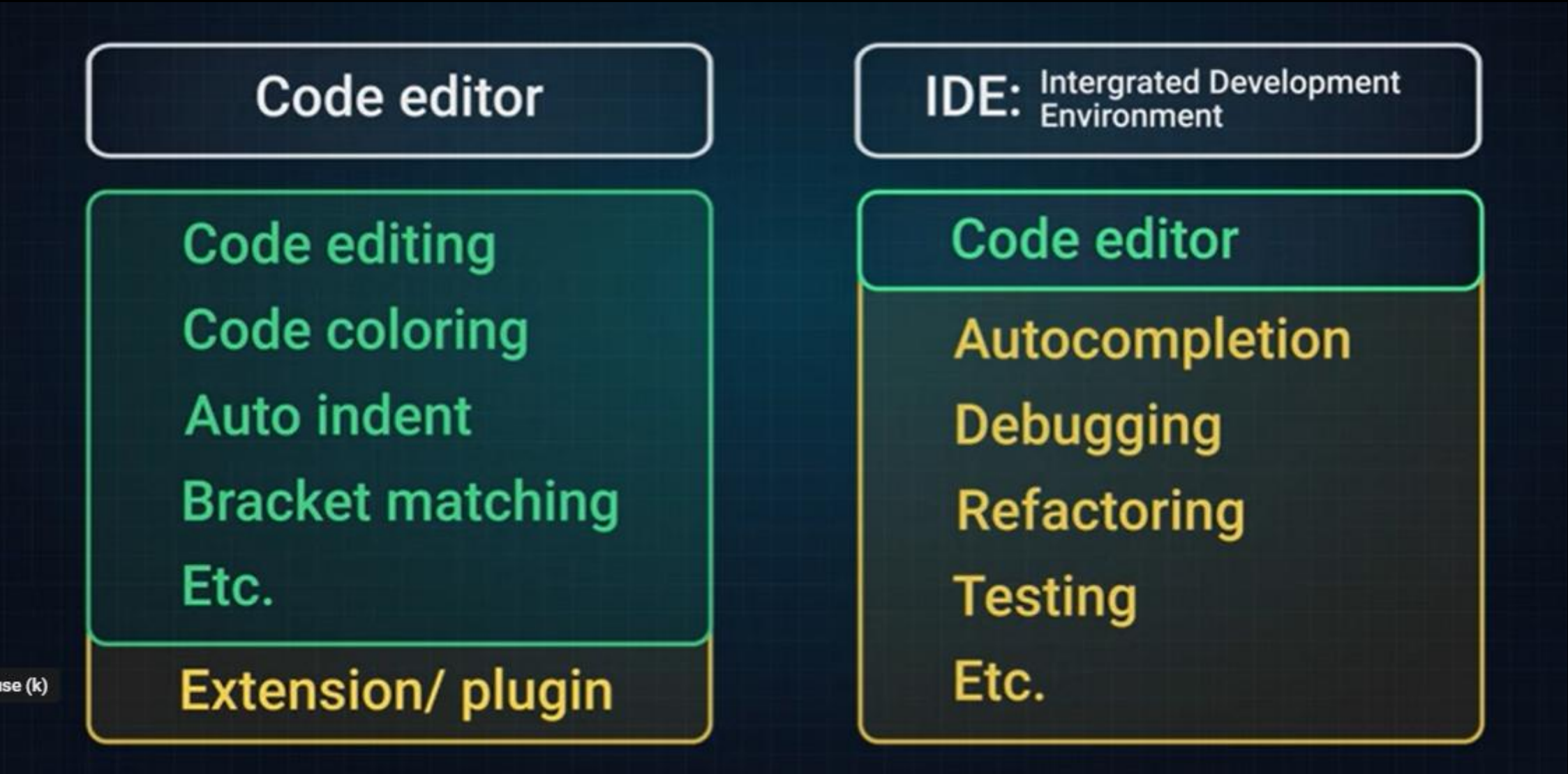
ing functions.

ments, keyword

s. [More about](#)

and integrate systems more effectively. [>>> Learn More](#)





<https://www.online-python.com/>





<https://code.visualstudio.com/download>



<https://www.jetbrains.com/pycharm/download>

<https://www.online-python.com/>



Python Course

hello.py

Extension: Python

python

isort

Import organization sup...

Microsoft

50ms

Python

Python language suppor...

Microsoft

232ms

Pylance

A performant, feature-ric...

Microsoft

421ms

Snippet

Insert a snippet from cht...

vscode-snippet

Python ...

Python Debugger extens...

Microsoft

65.1M

4.5

Install

Python In...

Correct Python indentati...

Kevin Rose

12.8M

4

Install

Python

Microsoft

python.python

155,625

Python language support with extension ...

Disable

Uninstall

Switch to Pre-Release

DETAILS

FEATURES

CHANGELOG

EXTENSION PACK

Python extension for Visual Studio Code

A Visual Studio Code extension with rich support for the Python language (for all actively supported Python versions) providing access points for extensions to seamlessly integrate and offer

Installation

Identifier	ms-python.python
Version	2025.0.0
Last Updated	2025-02-11, 15:20:48
Size	28.87MB
Cache	1.79KB

Marketplace

No Solution

BLACKBOX Chat

Add Logs

CyberCoder

Improve Code

Share Code Link

Quokka

PC

Python_Course_1

Version control

Project

Python_Course_1 C:\Users\HP\PycharmProjects\Python_Course_1

.venv library root

Include

Lib

Scripts

.gitignore

pyvenv.cfg

External Libraries

Scratches and Consoles

+

▶

≡

⏏

!

🔗

Current File

▶

⚙

⋮

👤

🔍

⚙

—

□

✕

🔔

🌀

🗄

Everywhere Double Shift

File Ctrl+Shift+N

Files Ctrl+E

tion Bar Alt+Home

les here to open them

Python_Course_1

Python 3.12 (Python_Course_1)

Free Educational Licenses

Learn or teach coding with best-in-class development tools from JetBrains!

For students and teachers

For schools and universities

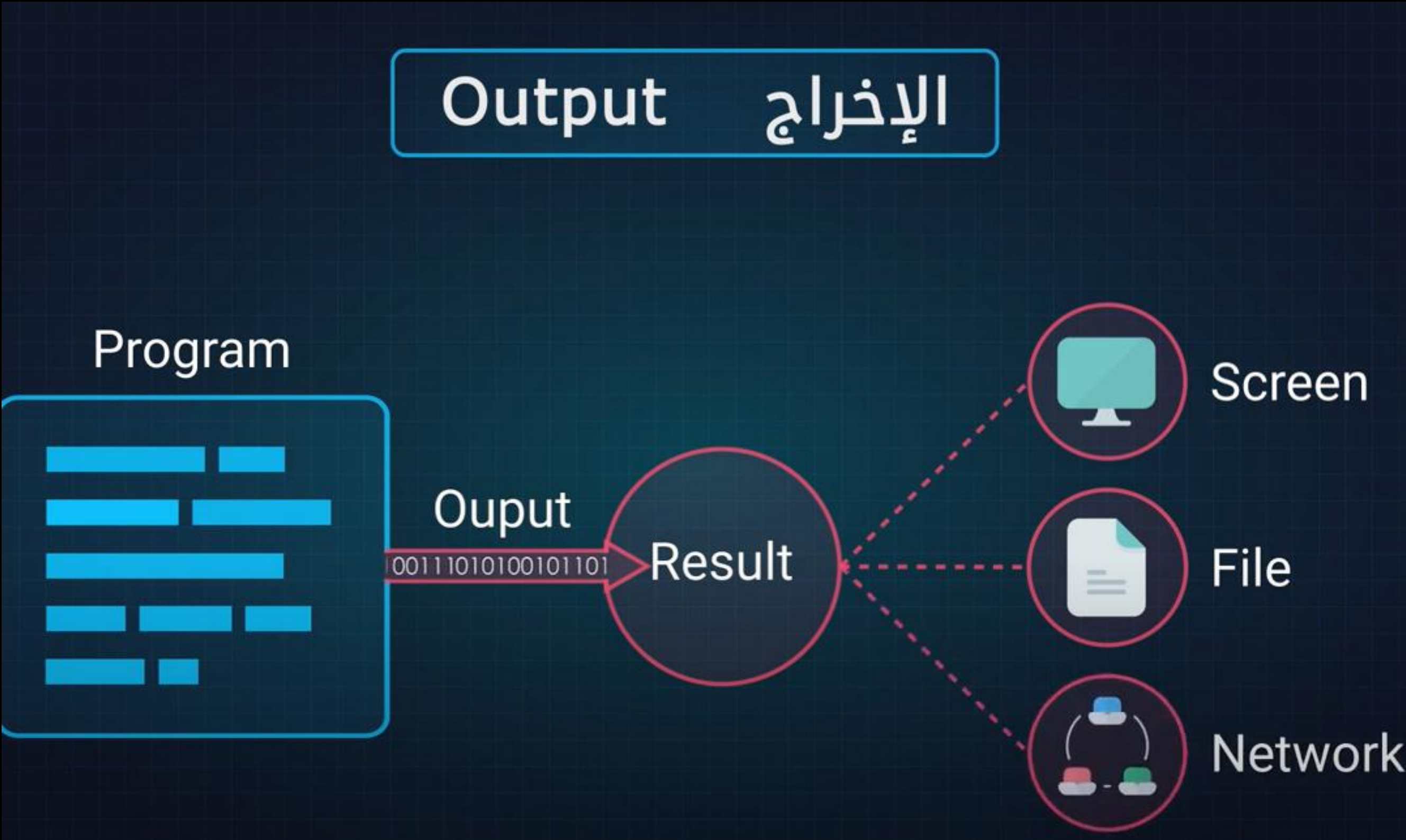
Individual licenses for students and teachers

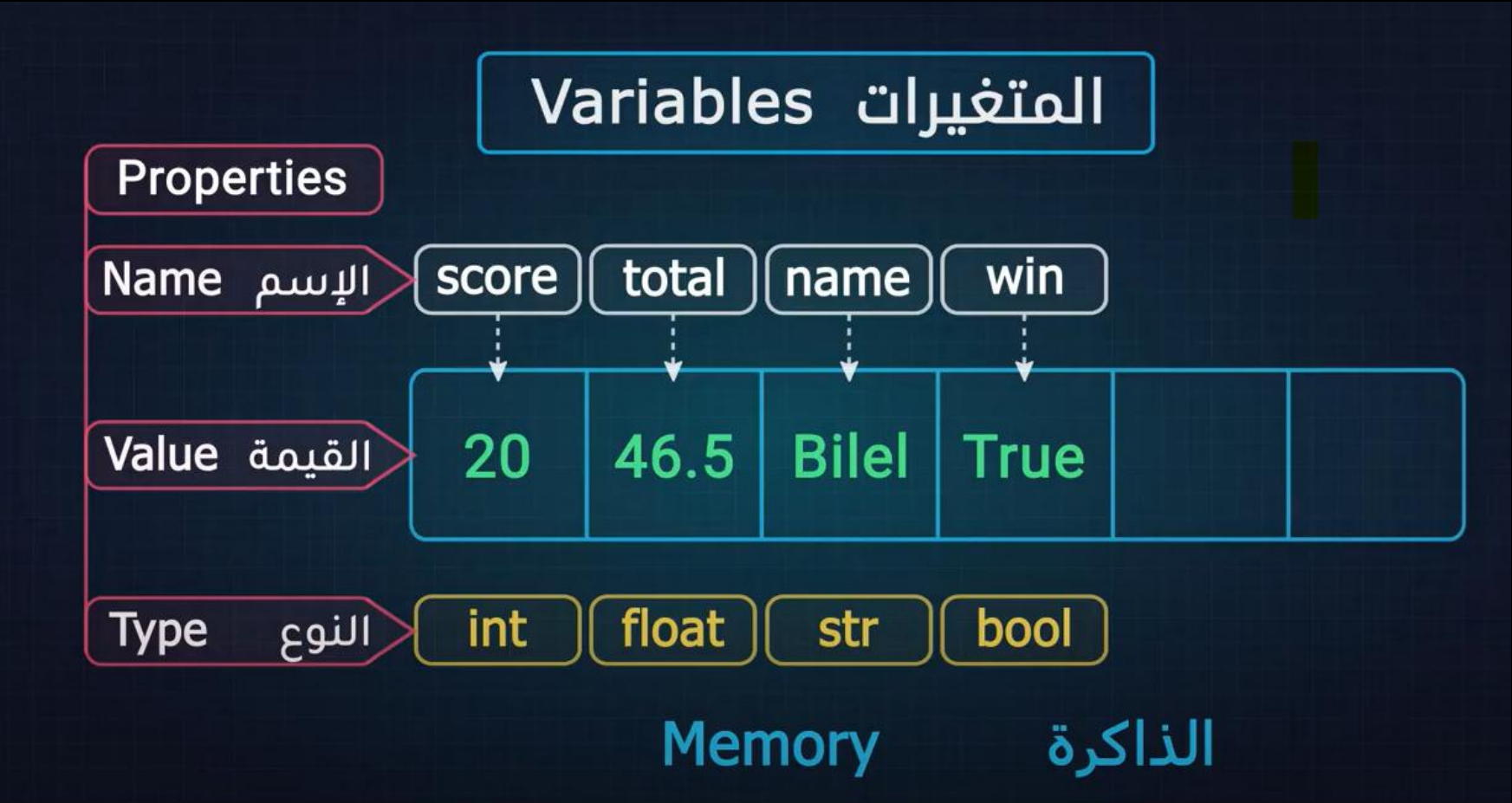
Get free access to all JetBrains IDEs for personal use at school or at home.

Here is a list of the Python keywords.

False	class	from	or
None	continue	global	pass
True	def	if	raise
and	del	import	return
as	elif	in	try
assert	else	is	while
async	except	lambda	with
await	finally	nonlocal	yield
break	for	not	

Practice





☒ True	☒ _x
☒ Grade75	☒ price\$
☒ Birth date	☒ 3color
☒ UserCategory	☒ student_name

input	Method	output
"Hello WORLD"	.capitalize()	Hello World
"HELLO WORLD"	.lower()	hello world
"hello world"	.upper()	HELLO WORLD
‘Python’	.center(10,‘*’)	**Python**
"HELLO WORLD"	.count(‘L’)	3
"HELLO WORLD"	.index(‘O’)	4
"HELLO WORLD"	.find(‘OR’)	7
‘31/01/2025’	.replace(‘/’, ‘-’)	‘31-01-2025’
‘31/01/2025’	.split(‘/’)	[‘31’, ‘01’, ‘2025’]
‘abc123’	.isalnum()	True
‘12345’	.isnumeric()	True
‘hello world’	.islower()	True
‘HELLO WORLD’	.isupper()	True

 **int**
age = 25

 **float**
score = 47.5

 **complex**
x = 1 + 2i

```
print(type(score))
```

 <class 'float'>

Python Syntax	Description	Example	Result
+	Addition	20 + 3	23
-	Substraction	20 - 3	17
*	Multiplication	20 * 3	60
/	Division	20 / 3	6.666
//	Integer division	20 // 3	6
%	Modulus	20 % 3	2

Python Syntax	Example	Equivalent
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
//=	x //= 3	x = x // 3
%=	x %= 3	x = x % 3
**=	x **= 3	x = x ** 3

Arithmetic Operators:-

Arithmetic operators in Python are used to perform mathematical operations between variables and values.

Operator	Description	Example
+	Adds two operands	5 + 3
-	Subtracts the second operand from the first	5 - 3
*	Multiplies two operands	5 * 3
/	Divides the first operand by the second (returns float)	5 / 3
//	Divides the first operand by the second (returns int)	5 // 3
%	Returns the remainder when the first operand is divided by the second	5 % 3
**	Raises the first operand to the power of the second	5 ** 3

Comparison operators:-

Comparison operators in Python are used to compare two values.

Operator	Name	Description	Example
==	Equal to	True if both operands are equal	5 == 3
!=	Not equal to	True if operands are not equal	5 != 3
>	Greater than	True if left operand is greater	5 > 3
<	Less than	True if left operand is smaller	5 < 3
>=	Greater than or equal to	True if left operand is greater or equal	5 >= 3
<=	Less than or equal to	True if left operand is smaller or equal	5 <= 3

Logical operators:-

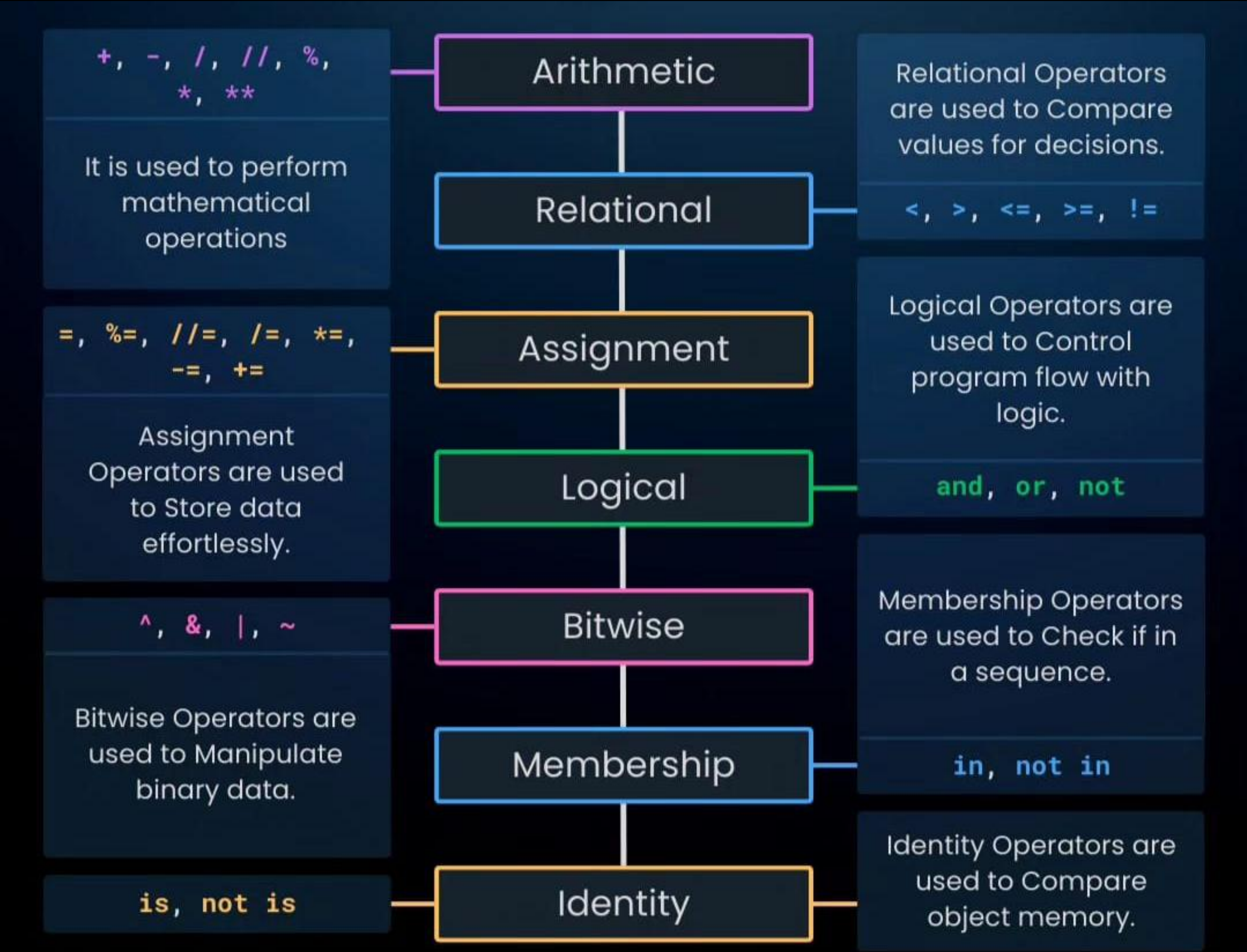
Logical operators in Python are used to combine conditional statements.

Operator	Description	Example	Result
and	True if both statements are true	True and False	False
or	True if at least one of the statements is true	True or False	True
not	Reverses the result, True becomes False, and vice versa	not True	False

Assignment Operators:-

Assignment operators in Python are used to assign values to variables, and they can also combine an operation with the assignment in a single step.

Operator	Name	Example	Equivalent
=	Assign	a = 5	
+=	Add and assign	a += 3	a = a + 3
-=	Subtract and assign	a -= 3	a = a - 3
*=	Multiply and assign	a *= 3	a = a * 3
/=	Divide and assign	a /= 3	a = a / 3
//=	Floor divide and assign	a //= 3	a = a // 3
%=	Modulus and assign	a %= 3	a = a % 3
**=	Exponentiate and assign	a **= 3	a = a ** 3



Python Membership Operators:-

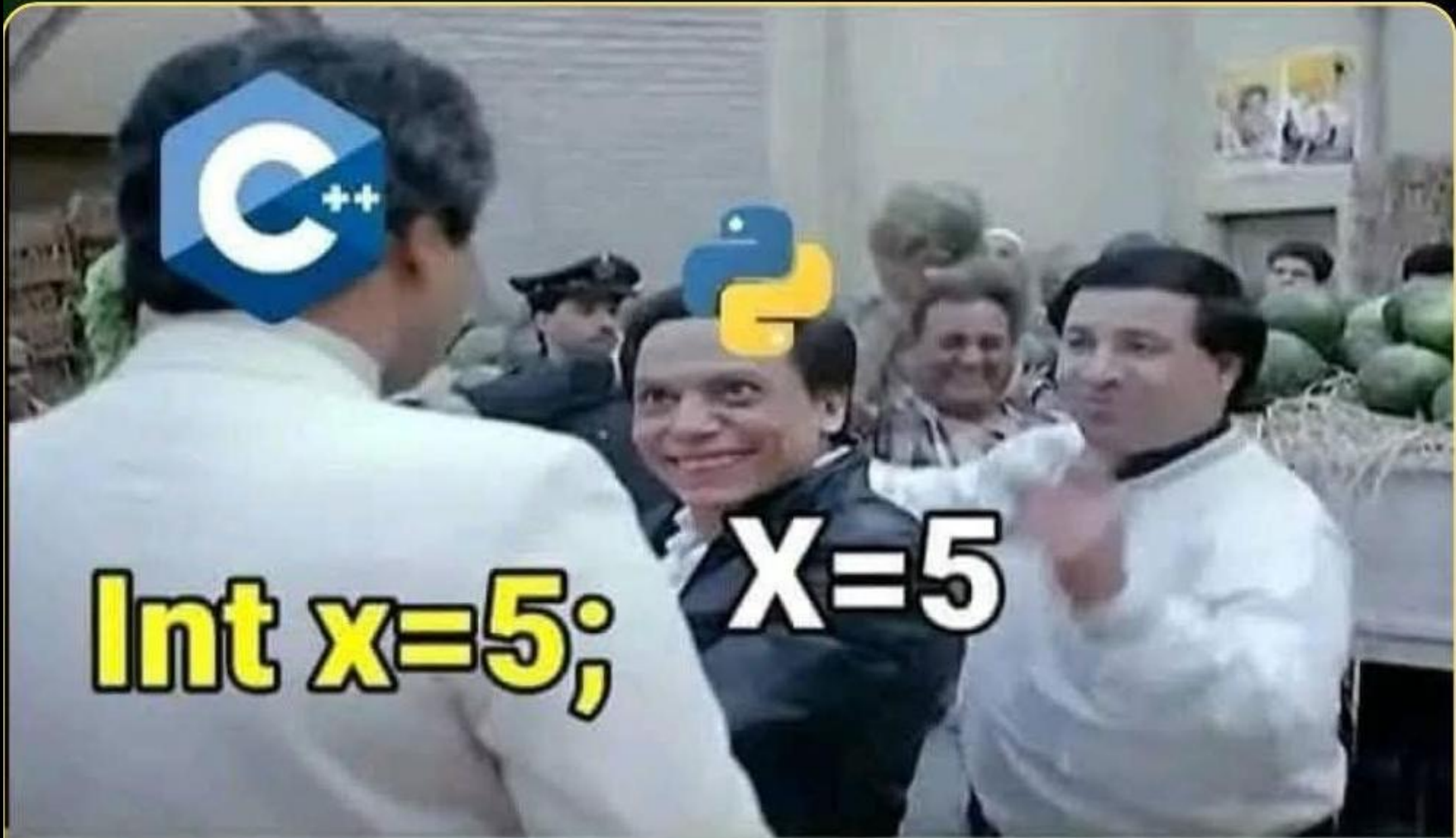
Membership operators are used to test if a sequence is presented in an object.

Operators	Description	Example
in	Returns True if a sequence with the specified value is present in the project	a in b
not in	Returns True if a sequence with the specified value is not present in the object	a not in b

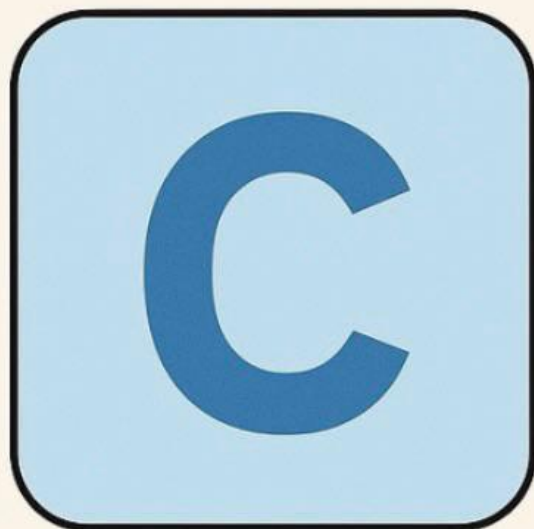
Python Identity Operators:-

Identify Operators are used to compare the objects not if they are equal, but if they are the same object, with the same memory location.

Operator	Description	Example
is	Return True if both variables are the same	a is b
is not	Return True if both variables are not the same objects	a is not b

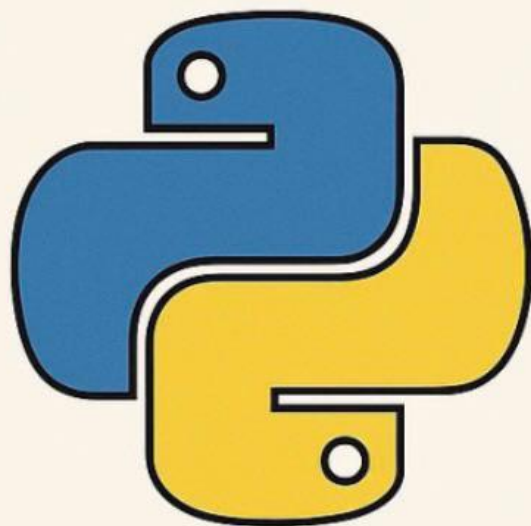


TWO VARIABLE SWAPPING IN C AND PYTHON



C LANGUAGE

```
int x, y, temp;  
temp = x;  
x = y;  
y = temp;
```



PYTHON

```
x, y = y, x
```



if

"if" statement allows you to execute a **block of code** if a certain condition is met.

```
x = 5
if x > 0:
    print("x is positive")
```

Output:

```
x is positive
```

else

"else" statement allows you execute a **different block of code** if the condition is not met:

```
x = -5
if x > 0:
    print("x is positive")
else:
    print("x is not positive")
```

Output:

```
x is not positive
```

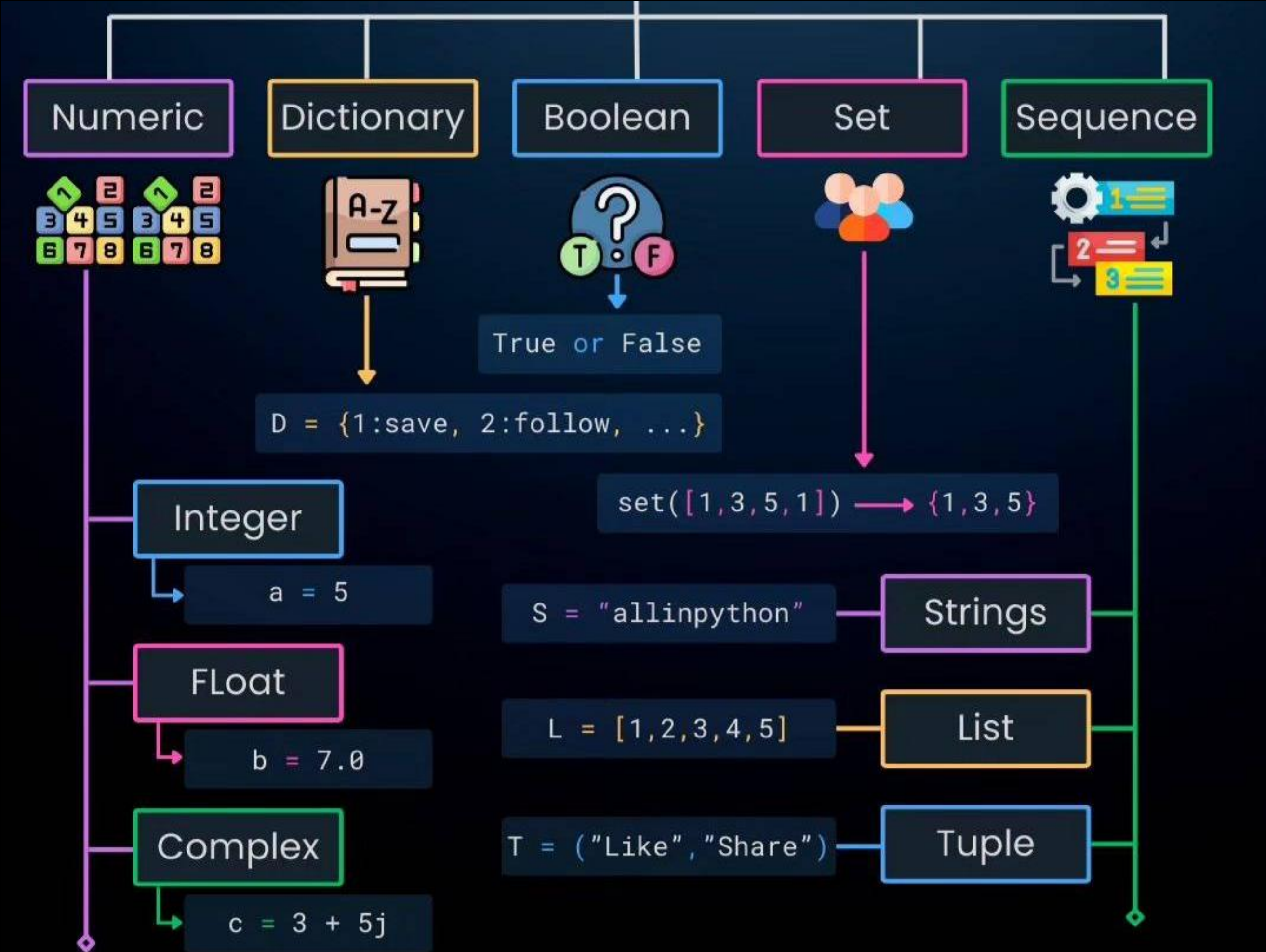
elif

Sometimes you might have **more than two possible outcomes**. In that case, you can use an **"elif"** statement to check additional conditions:

```
x = 0
if x > 0:
    print("x is positive")
elif x < 0:
    print("x is negative")
else:
    print("x is zero")
```

Output:

```
x is zero
```

There are different types of data in Python. Some built-in Python data types are:

Numeric data types: *int, float, complex*

String data types: *str*

Sequence types: *list, tuple, range, generator*

Binary types: *bytes, bytearray, memoryview*

Mapping data type: *dict*

Boolean type: *bool*

Set data types: *set, frozenset*



LIST

Definition

A list is a collection of items, which is ordered and mutable.

Syntax

```
list_name = [item1, item2, ...]
```

PYTHON LIST METHODS

@cydencode

<u>INPUT</u>	<u>METHOD</u>	<u>OUTPUT</u>
[10, 20, 30]	.append(40)	[10, 20, 30, 40]
[10, 20, 30]	.extend([40])	[10, 20, 30, 40]
[10, 20, 30]	.insert(2, 40)	[10, 20, 40, 30]
[10, 20, 30, 20]	.count(20)	2
[10, 20, 30]	.clear()	[]
[10, 20, 30, 40]	.index(40)	3
[10, 20, 30, 40]	.remove(10)	[20, 30, 40]
[10, 20, 30, 40]	.pop(2)	[10, 20, 40]
[30, 20, 10]	.reverse()	[10, 20, 30]
[30, 10, 40, 20]	.sort()	[10, 20, 30, 40]
[10, 20, 30]	.copy()	[10, 20, 30]

PYTHON METHODS CHEATSHEET

   `.append(  )` →    

   `.clear()`

   `.count(  )` → 2

   `.copy()` →   

   `.index(  )` → 2

   `.insert( 1,  )` →    

    `.pop( 3 )` →   

   `.remove(  )` →  

   `.reverse()` →   

Python List Methods

Input	Method	Output
  	<code>.append()</code>	   
  	<code>.insert(1, )</code>	   
   	<code>.pop(1)</code>	  
   	<code>.remove()</code>	  
  	<code>.reverse()</code>	  
  	<code>.sort()</code>	  
  	<code>.index()</code>	2
   	<code>.count()</code>	2

for

If you know in advance **how many times you want to repeat a block** of code, you can use a **"for"** loop:

```
for x in range(5):  
    print(x)
```

Output:

```
0  
1  
2  
3  
4
```

for

You can also use a **"for"** loop to **iterate over the items** of a list or any other iterable object:

```
fruits = ["apple", "banana", "cherry"]  
for fruit in fruits:  
    print(fruit)
```

Output:

```
apple  
banana  
cherry
```

while

"while" loop allows you to **repeat a block of code** as long as a certain condition is true:

```
x = 0  
while x < 5:  
    print(x)  
    x += 1
```

Output:

```
0  
1  
2  
3  
4
```

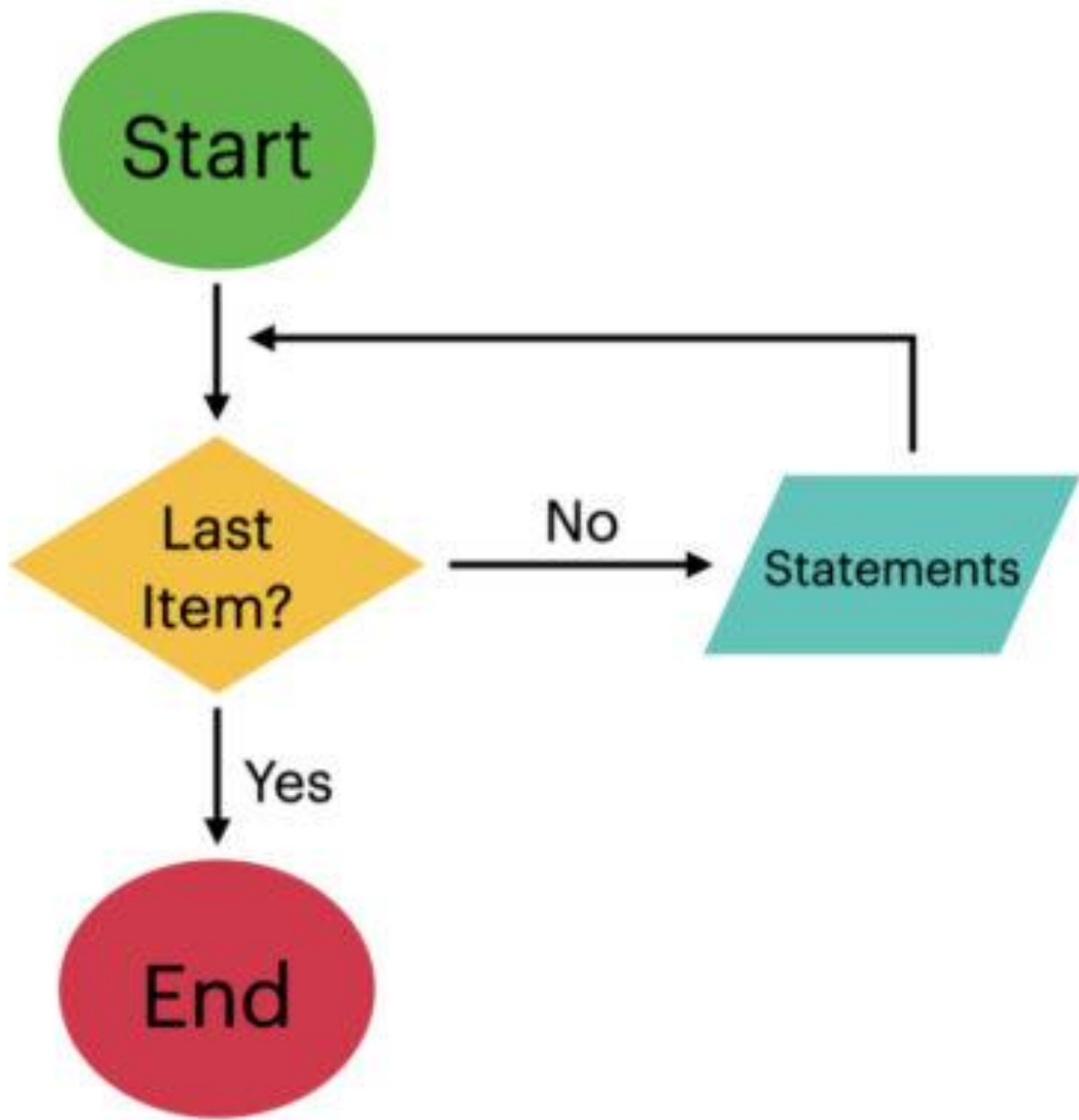



For loop

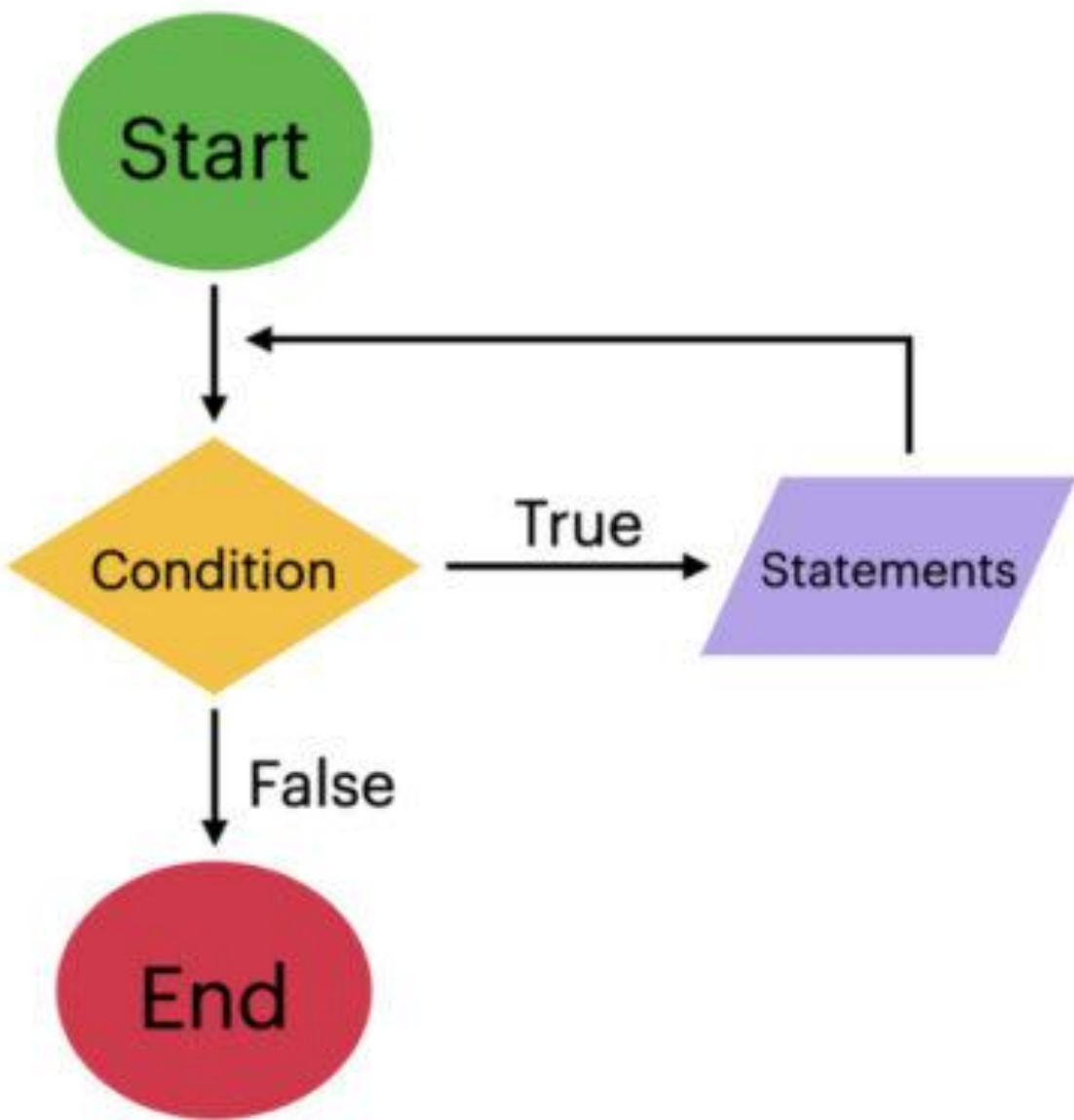


While loop

For Loop



While Loop



for

You can also use a "for" loop to iterate over the items of a list or any other iterable object:

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

Output:

```
apple
banana
cherry
```

continue

Finally, you can use the "continue" statement to skip the rest of the current iteration and move on to the next one:

```
for x in range(10):
    if x % 2 == 0:
        continue
    print(x)
```

Output:

```
1
3
5
7
9
```

break

Sometimes you might want to exit a loop early if a certain condition is met. You can use the "break" statement to do that:

```
x = 0
while True:
    if x == 5:
        break
    print(x)
    x += 1
```

Output:

```
0
1
2
3
4
```

For Loop

In Python

The diagram illustrates the components of a Python for loop. It features a code block with two lines of code. Above the code, three labels with arrows point to specific parts: 'Initialization' points to the variable 'i', 'Start a loop' points to the 'in' keyword, and 'Stop a loop' points to the second argument '11' in the range function. A bracket on the right side of the code block is labeled 'Block of code'.

```
1 for i in range(1,11):
2     print(f'number {i}')
```

Initialization

Start a loop

Stop a loop

Block of code

Functions in Python

How to define functions in Python 3

Introduction

A function is a block of instructions that performs an action and, once defined, can be reused. Functions make code more modular, allowing you to use the same code over and over again.

Python has a number of built-in functions that you may be familiar with, including:

- `print()` which will print an object to the terminal
- `int()` which will convert a string or number data type to an integer data type
- `len()` which returns the length of an object

Function names include parentheses and may include parameters (arguments). we'll go over how to define your own functions to use in your coding projects

Defining a Function

A function is defined by using the `def` keyword, followed by a name of your choosing, followed by a set of parentheses which hold any parameters the function will take (they can be empty), and ending with a colon.

```
1 def hello():  
2     print("Hello, World!")
```

Our function is now fully defined, but if we run the program at this point, nothing will happen since we didn't call the function.

```
1 def hello():  
2     print("Hello, World!")  
3 hello()
```


Working with Parameters

```
1 def add_numbers(x, y, z):
2     a = x + y
3     b = x + z
4     c = y + z
5     print(a, b, c)
6 add_numbers(1, 2, 3)
```

```
1 # Define function with parameters
2 def profile_info(username, followers):
3     print("Username: " + username + " has " + str(followers) + " Followers.")
4
5 name="Abdesselam"
6 followers = 340
7 profile_info(name, followers)
```

Returning a value

You can pass a parameter value into a function, and a function can also produce a value.

A function can produce a value with the return statement, which will exit a function and optionally pass an expression back to the caller. If you use a return statement with no arguments, the function will return None.

```
1  def square(x):  
2      y = x ** 2  
3      return y  
4  
5  result = square(3)  
6  print(result)
```